

10. Expectation-Maximization Algorithm

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

data = pd.DataFrame({
    "Age": [25,35,45,23,52,31,41,28,55,38,29,48],
    "Income":
[25000,50000,75000,22000,90000,45000,65000,30000,95000,55000,28000,80000],
    "Loan":
[10000,20000,30000,15000,25000,18000,22000,12000,20000,25000,10000,30000],
    "Payment_History": [1,1,1,0,1,1,1,0,1,1,0,1],
    "Credit_Score": ["Poor","Good","Excellent","Poor","Excellent",
        "Good","Excellent","Poor","Excellent","Good",
        "Poor","Excellent"]
})

encoder = LabelEncoder()
data["Credit_Score"] = encoder.fit_transform(data["Credit_Score"])

X = data[["Age", "Income", "Loan", "Payment_History"]]
y = data["Credit_Score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Actual:", y_test.values)
print("Predicted:", y_pred)

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

OUTPUT:

```
Data:
[ 1  2  3  4  5 10 11 12 13 14]

Cluster Labels:
[0 0 0 0 0 1 1 1 1 1]

Cluster Means:
[ 3.00001322 11.99998678]

Cluster Probabilities:
[[1.00000000e+00 1.54281120e-13]
 [1.00000000e+00 2.28973485e-11]
 [9.99999998e-01 2.06115362e-09]
 [9.99999814e-01 1.85539102e-07]
 [9.99989870e-01 1.01299910e-05]
 [1.01299910e-05 9.99989870e-01]
 [1.85539102e-07 9.99999814e-01]
 [2.06115362e-09 9.99999998e-01]
 [2.28973485e-11 1.00000000e+00]
 [1.54281120e-13 1.00000000e+00]]
```